OWASP API Security Risks

APIs fail in their own ways

Watch the 4-min video

In one line: APIs expose raw objects to scripts, so their top risks are object-level authorization (BOLA), weak authentication, returning or accepting too many properties, and unlimited consumption; fix them with per-object checks and explicit schemas.

1 Why a separate list?

APIs carry most traffic in modern apps. They return raw objects, accept raw JSON, and are called by scripts rather than browsers, so there is no UI to hide anything behind. OWASP keeps a separate **API Security Top 10**; the current edition is **2023**.

2023 risk	In one line
API1 Broken Object Level Authorization	no check that the caller may access *this* object ID
API2 Broken Authentication	weak login, unchecked or never-expiring tokens
API3 Broken Object Property Level Authorization	too many fields returned (excessive data exposure) or accepted (mass assignment)
API4 Unrestricted Resource Consumption	no rate, size, page or cost limits
API5 Broken Function Level Authorization	regular users can call admin functions
API6 Unrestricted Access to Sensitive Business Flows	bots automate buying, sign-ups, reviews
API7 Server Side Request Forgery	API fetches attacker-chosen URLs (lesson 50)
API8 Security Misconfiguration	verbose errors, permissive CORS, missing TLS
API9 Improper Inventory Management	forgotten old versions, test hosts, shadow APIs
API10 Unsafe Consumption of APIs	trusting data from third-party APIs

2 BOLA: the most common API bug

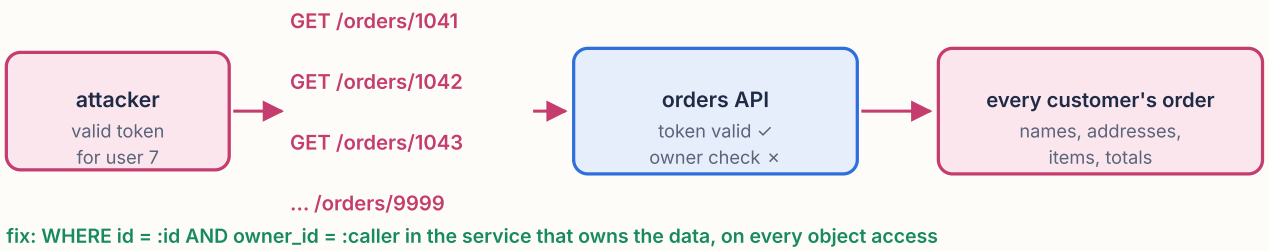


Figure 1. BOLA: authentication passes, so the attacker simply loops over IDs. Random IDs slow this down but don't fix it.

The API takes an ID from the path and returns the object without checking ownership (lesson 39). Fix it in the service that owns the data, on **every** access, including nested routes (`/users/7/orders/1042`), batch

endpoints and GraphQL resolvers. Automated tests should replay each request with a second user's token.

3 Broken authentication

Weak login flows, JWTs whose signature or `aud` is never checked (lesson 41), tokens that never expire, login and password-reset endpoints with no rate limit, and API keys sent in URLs where they end up in logs. Use a proven identity provider, validate every token fully, and keep tokens short-lived.

4 Too many properties: out and in

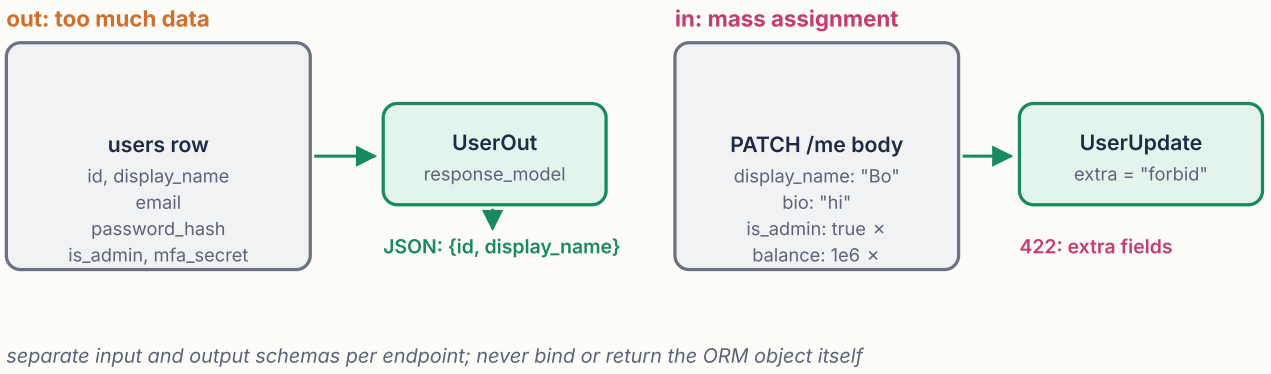


Figure 2. Excessive data exposure and mass assignment are two sides of API3:2023. Explicit schemas fix both.

Excessive data exposure: the API returns whole database objects and trusts the client to hide fields. The app shows a name; the JSON also has the email, password hash and internal flags, readable with any proxy. **Mass assignment** is the reverse: the API binds the request body onto the model, so a user adds `"is_admin": true` or a new balance. Rails, Django REST Framework and Spring all have this trap.

5 Limits and inventory

Unrestricted resource consumption: limit requests per user, key and IP (lesson 52), and also page sizes, upload sizes, query depth for GraphQL, and calls that cost you money (SMS, email). **Improper inventory**: forgotten `/v1` endpoints, staging hosts with production data, undocumented internal APIs. Keep a catalog generated from OpenAPI specs and gateway logs, and retire old versions on purpose.

WORKED EXAMPLE: FASTAPI RESPONSE AND INPUT MODELS

`class UserOut(BaseModel): id: int; display_name: str` is declared as `response_model=UserOut`, so FastAPI filters the returned object to exactly those fields. Updates use `class UserUpdate(BaseModel): display_name: str; bio: str` with `model_config = {"extra": "forbid"}`, so a body containing `is_admin` is rejected with 422. The handler then copies only validated fields onto the row. Over-exposure and mass assignment are blocked by design, not by memory.

COMMON MISTAKES

- Returning `user.__dict__` or the ORM object and filtering in the mobile app.
- Checking the user's role for the endpoint but not ownership of the object.
- Rate limiting only by IP, while attackers rotate through thousands of IPs.
- Leaving `/v1` running “for a few clients” without the fixes made in `/v2`.

6 Key terms

BOLA: API returns an object without an ownership check

Excessive data exposure: response includes fields the client shouldn't see

Response / input model: explicit schema of allowed output / input fields

BFLA: regular user can call an admin function

Mass assignment: request sets fields it shouldn't, like is_admin

Shadow API: endpoint missing from the inventory, so unpatched

7 Interview questions

Q1. What's the top API risk? BASIC

Broken object level authorization: always check that the caller may access that specific object, in the service that owns it.

Q2. What is mass assignment? BASIC

Binding a request body directly onto a data model, so a client can set fields it shouldn't (is_admin, balance). Prevent it with explicit input schemas that allow only safe fields and reject extras.

Q3. A mobile app shows only names, but a pentester finds emails and password hashes in the JSON. Fix? SCENARIO

Return a response model with only the fields that screen needs, never the database object; audit other endpoints the same way and remove hashes from any API layer.

Q4. How do you find APIs you didn't know you had? SCENARIO

Build an inventory from gateway and load-balancer logs, DNS records and OpenAPI specs; flag hosts and versions with traffic but no owner, then retire or secure them.

Q5. BOLA vs BFLA: what's the difference? TRICKY

BOLA is access to another user's *object* through a permitted function; BFLA is calling a *function* you shouldn't (e.g. DELETE /admin/users) at all.

Q6. Do UUIDs instead of sequential IDs fix BOLA? TRICKY

No. They stop easy enumeration, but IDs leak through links, logs and other responses. Only an authorization check on each object access fixes it.

Next: SD-M-014 · Abuse Prevention backendroadmap.com